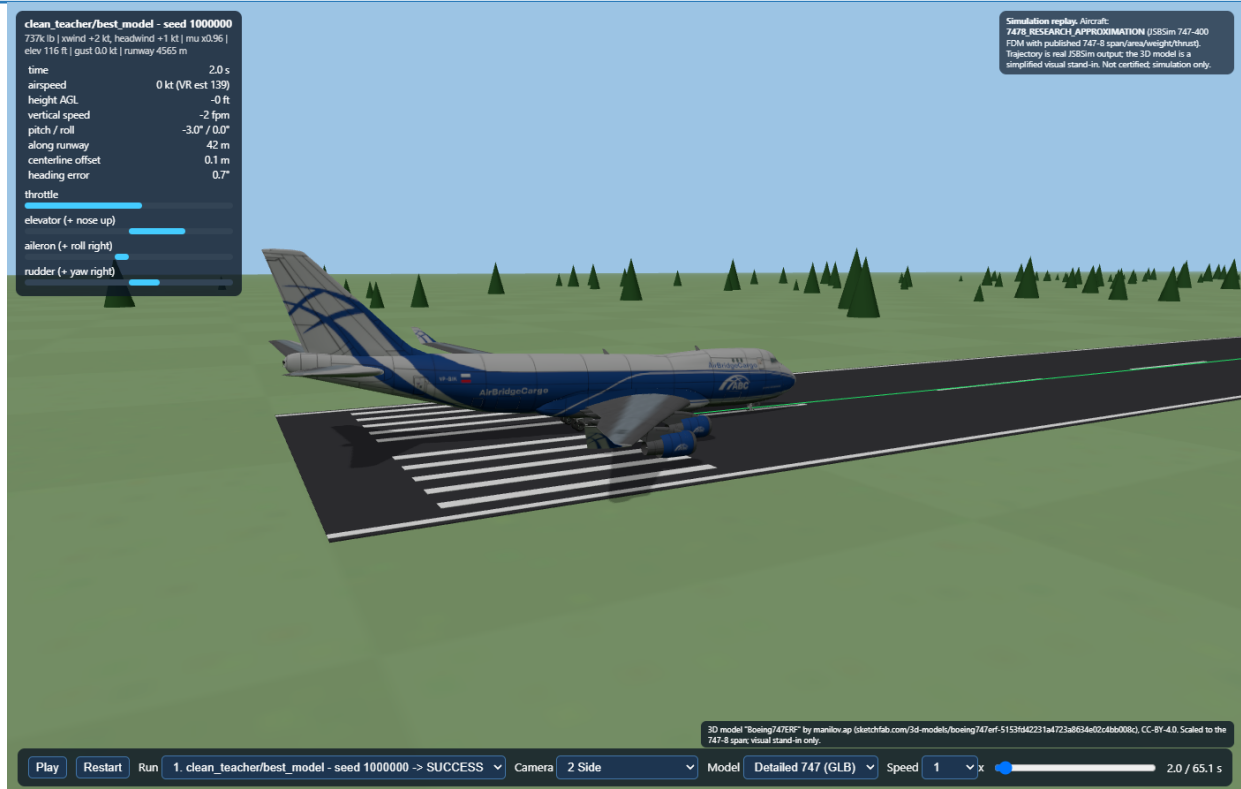


Teaching a Giant Airliner to Take Off

A friendly guide to the Boeing 747-8 reinforcement-learning project

Project handover snapshot: 2026-10-08 11:34 Maroc

One sentence: A simulated pilot learns to control throttle, elevator, aileron and rudder so a research approximation of a Boeing 747-8 can perform a clean, stable takeoff in many changing conditions.



The project's 3D replay viewer. The detailed aircraft is a visual stand-in; the flight physics come from JSBSim.

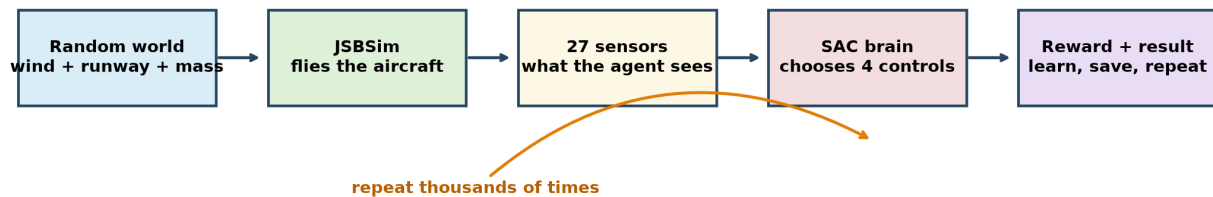
How to use this report

Read pages 2-4 to understand the idea. Read the current-stage pages to see what works today. Use the final checklist when you want to continue training or evaluation.

1. The one-minute story

Imagine a student pilot practising the same takeoff thousands of times. The pilot sees 27 instrument values, moves four controls, and receives a score. Good actions earn points; dangerous or messy actions lose points. After every attempt, SAC adjusts a neural network so the next attempt can improve. A hand-written teacher supplies good examples so learning does not start from pure chaos.

The whole project in one picture

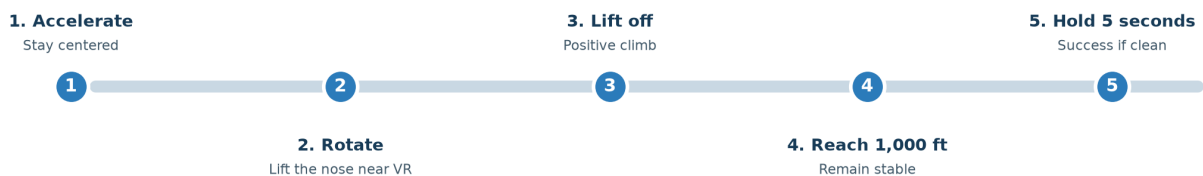


The learning loop. Remember: world - physics - sensors - brain - score.

2. What one successful takeoff must do

Success is stricter than simply leaving the runway. The aircraft must pass the complete sequence below and remain clean enough over the whole trajectory.

What one successful episode must do



The five-step takeoff story.

3. The cast of characters

Word	Plain-English meaning
Agent	The neural-network pilot.
Environment	The practice world that runs and judges the flight.
Observation	The 27-number instrument panel.
Action	Throttle, elevator, aileron and rudder.
Reward	A score saying 'more like this' or 'less like this'.
Episode	One complete takeoff attempt.
Teacher	A conventional controller that demonstrates sensible takeoffs.
Curriculum	Five levels that make conditions harder.
Checkpoint	A saved brain, replay buffer and training state.

4. The aircraft: useful, but not certified

The physics engine is JSBSim. No validated JSBSim 747-8 model was found, so the project starts from JSBSim's Boeing 747-400 and replaces a limited set of public numbers: wingspan, wing area, empty weight, fuel capacity and engine thrust. Aerodynamics, inertia, gear and many engine details still come from the 747-400 model. The name 7478_RESEARCH_APPROXIMATION is an honest warning: simulation research only.

5. What the agent sees and controls

The 27 observations cover speed, height, climb rate, attitude, runway position, control positions, engine state, mass, centre of gravity, wind and air density. Values are normalised and clipped before the neural network sees them.

The action vector contains four values in $[-1,1]$. An actuator layer clips invalid commands, rate-limits movement, maps throttle to $[0,1]$, fixes JSBSim sign conventions and connects rudder to nose-wheel steering. The policy acts at 10 Hz; JSBSim performs 12 physics steps at 120 Hz for each decision.

6. The score: fast, straight, smooth, stable

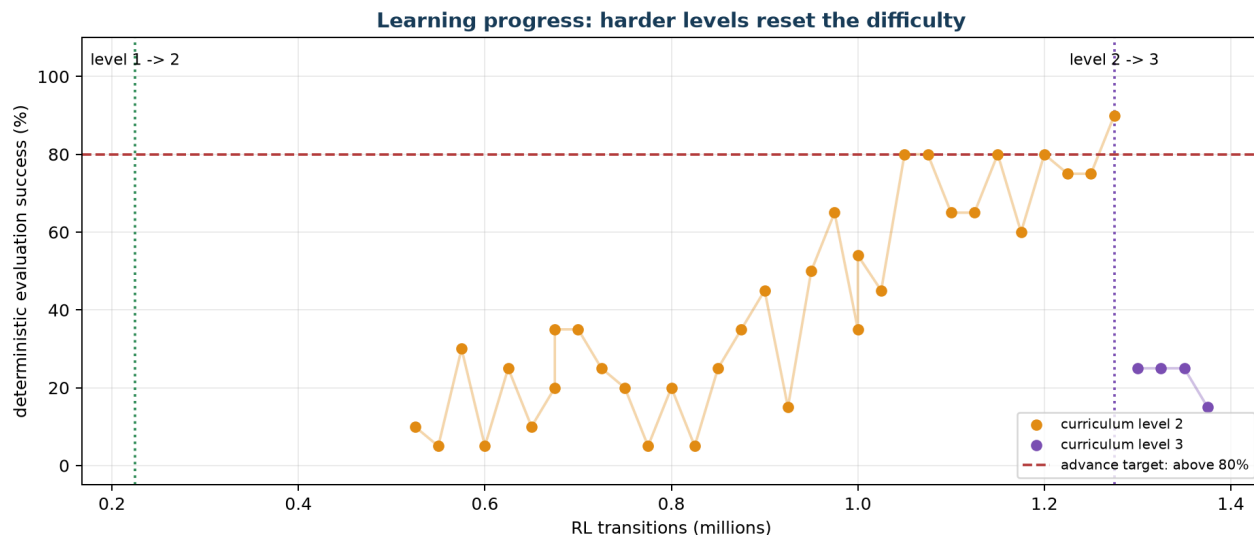
Positive reward comes from progress, centreline tracking, correct heading, acceleration, rotation, climb, altitude, stability and success. Penalties cover time, lateral error, excessive attitude, rough controls, premature rotation and failures. The clean profile adds passenger-comfort terms for tight centreline tracking, low sideways acceleration, low yaw/roll rate and a clean-takeoff bonus.

FAST ENOUGH - STRAIGHT ENOUGH - SMOOTH ENOUGH - STABLE ENOUGH

7. How the pilot learns

The main algorithm is Soft Actor-Critic (SAC). Its actor and critics use three 256-unit ReLU layers. It learns from a replay buffer of one million transitions. Teacher demonstrations enter the buffer first, and behaviour cloning gives the actor a sensible starting point. SAC then improves through its own exploration.

Curriculum advancement depends on deterministic evaluation, not noisy training reward. The policy advances when evaluation is above 80 percent with at least 20 episodes. Each new level makes the exam harder.

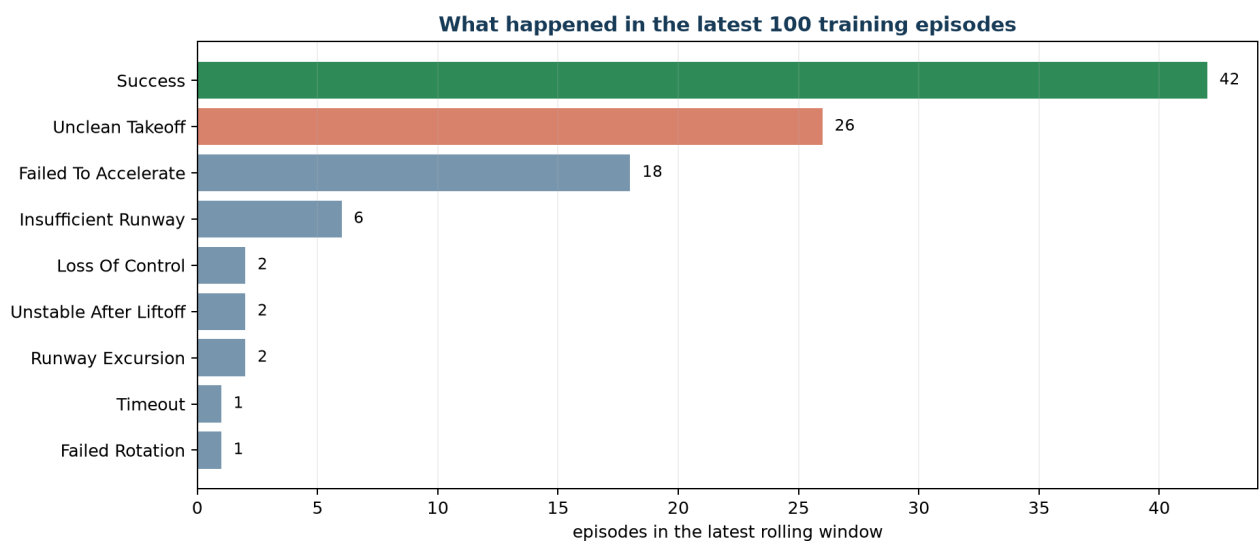


Evaluation progress. Drops after level-ups are expected because the conditions become harder.

8. Current stage

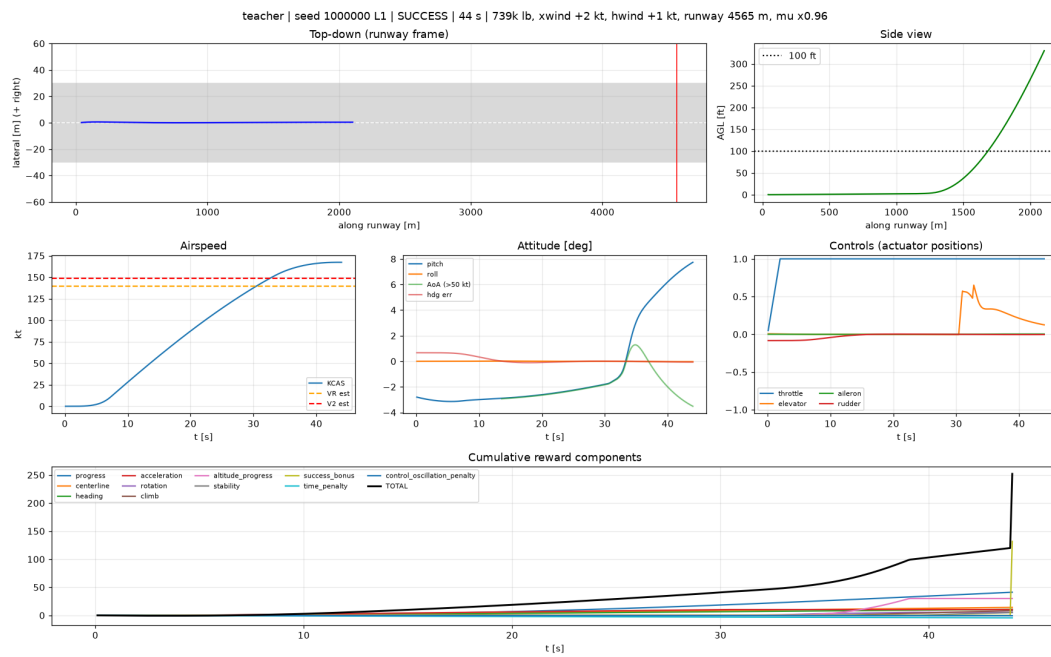
Training was active when this report was built, so these numbers are a dated snapshot. The system has learned convincing behaviour through level 2 and is now adapting to the harder level-3 conditions. It is not yet a final, fully robust policy.

Snapshot item	Value
Active run	clean_teacher_lv4
RL transitions	1,386,000
Curriculum level	3
Training episodes	920
Training successes	585
Rolling success (latest 100)	42%
Latest deterministic evaluation	15% at level 3
First success	107,676 transitions
Level 1 -> 2	225,000 transitions
Level 2 -> 3	1,275,000 transitions



Latest 100 training outcomes. Unclean takeoff means the target region was reached, but the complete trajectory was not smooth enough.

9. What good and bad flights look like



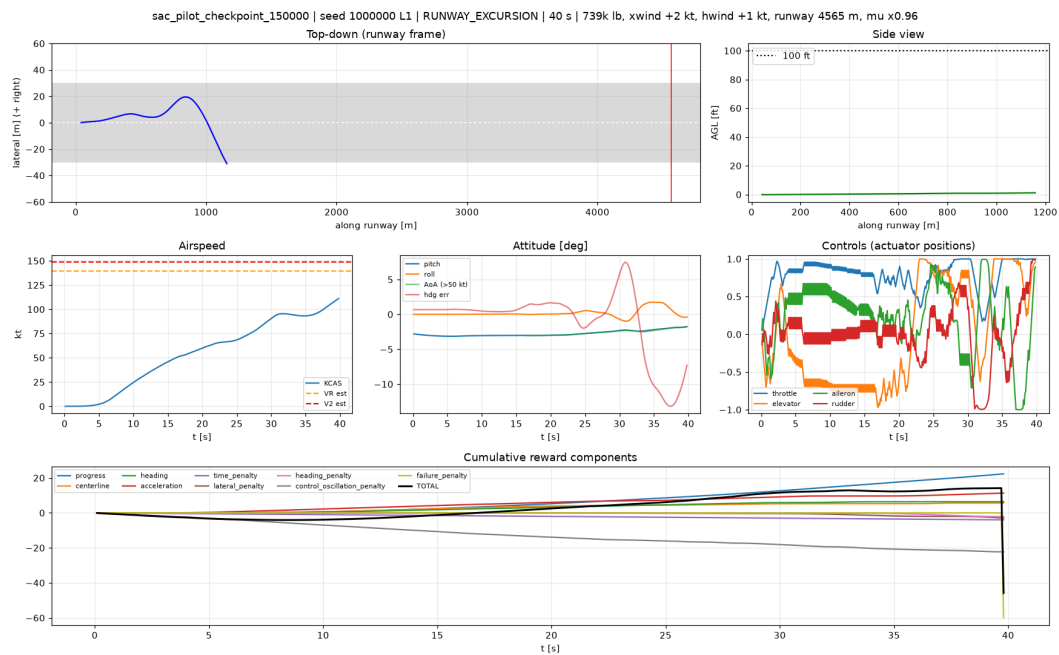
SUCCESS: the dashboard shows a centered runway track, a smooth rotation and a stable climb. A beginner should check that the centerline error stays small, the roll stays controlled and the climb continues.



SUCCESS IN 3D: a recorded training success viewed from behind. The aircraft begins on the runway centerline, and the green event list confirms repeated successful episodes.



SUCCESS CHECKPOINT: this side view lets us check correct wheel contact, runway alignment and aircraft attitude at the start of the replay.



FAILURE: an early learned policy wanders away from the centerline, moves the controls roughly and eventually leaves the runway. This tells the next developer to inspect centerline error, heading error and control smoothness.

How to read these pictures

Use three quick questions: (1) Is the aircraft close to the runway centerline? (2) Are heading and roll small and controlled? (3) After rotation, does altitude rise steadily without wild control movement? A success answers yes to all three. A failure picture is useful evidence: it points to the observation, reward or controller behaviour that needs investigation.

10. Project map

Path	Responsibility
configs/	Default aircraft, environment, reward, randomisation and learning settings.
configs_975/	Special 975,000-lb experiment configuration.
envs/	Gymnasium environment, JSBSim bridge, observations, actions, reward and termination.
training/	SAC/PPO entry points, callbacks, checkpoints and curriculum.
teacher/	Teacher controller and demonstrations.
evaluation/	Evaluation, robustness, failure analysis and trajectory recording.
tools/	Checks, benchmarks, plots and 2D/3D viewers.
tests/	Unit, integration and showcase tests.
models/	Policy ZIPs, replay-buffer PKLs and state JSON files.
results/	Metrics, milestones, CSVs, demonstrations and replay events.
logs/	TensorBoard and console logs.
msfs_sac_runtime.py	Experimental SimConnect bridge to Microsoft Flight Simulator.

11. Safe reproduction

```
cd C:\Users\asus\Documents\project\boeing7478_rl_takeoff
.venv\Scripts\Activate.ps1
python tools\check_environment.py
python -m pytest
tensorboard --logdir logs
python tools\live_viewer.py --open
```

To resume, keep the same reward profile and use the matching model, replay buffer and state file:

```
python -m training.train_sac --config configs/sac.yaml ^
--run-name clean_teacher_lv4 ^
--resume models/clean_teacher_lv4/checkpoint_1350000.zip ^
--reward-profile clean --total-timesteps 2000000
```

Stop or pause the active trainer before starting another process that writes to the same run or generated aircraft directory.

12. Validation status

The project environment checker passed all 10 checks: aircraft loading, shapes, action timing, bounded observations, rate-limited actions, deterministic seeds, reward arithmetic, terminations, random-action stability and Gymnasium validation.

The full pytest run executed 79 tests: 72 passed and 7 showcase tests could not complete because the active trainer held the generated aircraft XML while Windows also denied pytest's temporary directory. These are environment/locking errors rather than seven confirmed behaviour failures. Rerun pytest after stopping training.

13. Honest limitations

The aircraft is an approximation. There is no validated 747-8 aerodynamic database, no Boeing takeoff chart comparison, no runway slope, simplified friction, fuel fixed at the CG, and a visual model that does not affect physics. Level-3 evaluation is still low. The MSFS bridge fills several unavailable telemetry values with neutral defaults, so transfer to Microsoft Flight Simulator is experimental and must remain simulation-only.

14. Continuation checklist

1. Let the current run continue on level 3, but judge progress using deterministic evaluation.
2. Protect each checkpoint trio: model ZIP, replay-buffer PKL and state JSON.
3. After training stops, rerun the 10/10 checker and the full pytest suite.
4. Compare the learned policy with teacher and full-throttle baselines on identical seeds.
5. Investigate the dominant level-3 outcomes: unclean takeoff and failed acceleration.
6. Run 1000+ robustness episodes only after performance stabilises across harder levels.
7. Treat level 5 as a special slippery-runway stress test.
8. Improve the MSFS telemetry mapping before testing transfer.
9. Refresh this report after the next major checkpoint or curriculum level-up.

SEE -> STEER -> SCORE -> SAVE

27 observations -> 4 controls -> reward and outcome -> checkpoint and replay

15. Final handover

The project is currently a capable research simulator with a teacher-guided SAC policy that has graduated to curriculum level 3. A new contributor can continue safely by preserving experiment state, evaluating on fixed seeds, rerunning tests after the trainer stops, and improving robustness before making any claim beyond simulation.

Primary project sources

README.md; AIRCRAFT_MODEL_NOTES.md; EXPERIMENT_PROTOCOL.md; YAML files under configs/ and configs_975/; Python modules under envs/, training/, teacher/, evaluation/ and tools/; and the current status, milestone and CSV files under results/clean_teacher_lv4/.